P3568R2

`break label;` and `continue label;`

Published Proposal, 2026-05-10

This version:
https://eisenwave.github.io/cpp-proposals/break-continue-label.html

Author:
Jan Schultke

Audience:
SG22, EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
eisenwave/cpp-proposals

Abstract

Introduce `break label` and `continue label` to `break` and `continue` out of nested loops and `switch`es as accepted into C2y, and relax label restrictions.

Table of Contents

§ 1. Revision history

§ 1.1. Since R1

- Added some more motivation in § 3.4 Expansion statements
- Rebased § 7 Proposed wording on the post-Croydon working draft

§ 1.2. Since R0

The paper was seen by EWGI and then by EWG at Hagenberg 2025, with the following polls:

> **P3568R0**: EWG likes syntax `N3355: for (...) { }`
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 4 | 16 | 5 | 9 | 5 |
>
> **P3568R0**: EWG likes syntax `for N3377 (...) { }`
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 7 | 13 | 5 | 5 | 8 |
>
> **P3568R0**: If C has it, we are interested in this feature too.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 16 | 21 | 5 | 2 | 1 |

Due to lack of consensus in EWG, syntactical design choices were delegated to WG14. WG14 saw [N3377] at Graz 2025 and voted as follows:

> **N3377**: Would WG14 like to see a paper changing loop name syntax at a future meeting?
>
> | F | N | A |
> |---|---|---|
> | 6 | 11 | 9 |

The authors of [N3377] have expressed that they are no longer pursuing the paper; therefore, R1 of this paper assumes that the debate on label syntax is entirely settled, and C++ follows the [N3355] syntax.

Furthermore, the proposed wording has been improved slightly, and a `__cpp_break_label` feature-test macro is now included.

§ 2. Introduction

While C++ already has a broad selection of control flow constructs, one construct commonly found in other languages is notably absent: the ability to apply `break` or `continue` to a loop or `switch` when this isn't the innermost enclosing statement. This feature is popular, simple, and quite useful:

Specifically, we propose the following functionality:

```
outer: for (auto x : xs) {
    for (auto y : ys) {
        if (/* ... */) {
            continue outer; // OK, continue applies to outer for loop
            break outer;    // OK, break applies to outer for loop
        }
    }
}

switch_label: switch (/* ... */) {
    default: while (true) {
        if (/* ...*/) {
            break switch_label; // OK, break applies to switch, not to whi
        }
    }
}

break outer;       // error: cannot break loop from the outside
goto outer;        // OK, used to be OK, and is unaffected by this proposa

switch_label:;     // OK, labels can be reused
goto switch_label; // error: jump target is ambiguous
```

The `break label` and `continue label` syntax is identical to that in [N3355] and has been accepted into C2y (see working draft at [N3435]). We bring that syntax into C++ and relax restrictions on labels to make it more powerful, and to address concerns in a follow-up proposal [N3377].

Note that `break` and `continue` with labels have been proposed in [N3879] and rejected at Rapperswil 2014 ([N4327]):

> Straw poll, proposal as a whole:
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 1 | 1 | 1 | 13 | 10 |
>
> "break label;" + "continue label;"
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 3 | 8 | 4 | 9 | 3 |

I believe that rejecting `break label` was a grave mistake at a time. Regardless, the WG21 sentiment towards the feature is now the opposite, even if just for C compatibility (see § 1.2 Since R0).

§ 3. Motivation

`break label` and `continue label` are largely motivated by the ability to control nested loops. This is a highly popular feature in other languages, and C++ could use it too, since it has no good alternative.

To be fair, a conditional `return` in the loop sometimes bypasses the need to terminate it. However, this is not always allowed; such practice is outlawed by MISRA-C++:2008 Rule 6-6-5 "A function shall have a single point of exit at the end of the function" ([MISRA-C++]). Even if it is permitted, there are many cases where an early `return` does not obsolete `break`, and it generally does not obsolete `continue`.

> NOTE: I have been told that more recent revisions of MISRA-C++ no longer include this rule.

§ 3.1. No good alternative

Let's examine a motivating example which uses our new construct:

EXAMPLE 1 ¶

```
void f() {
    process_files: for (const File& text_file : files) {
        for (std::string_view line : text_file.lines()) {
            if (makes_me_angry(line)) {
                continue process_files;
            }
            consume(line);
        }
        std::println("Processed {}", text_file.path());
    }
    std::println("Processed all files");
}
```

`continue label` is very useful in this scenario, and expresses our intent with unparalleled clarity. We want to continue processing other files, so we `continue process_files`.

A plain `break` cannot be used here because it would result in executing the following `std::println` statement, but this should only be done upon success.

There are alternative ways to write this, but all of them have various issues.

§ 3.1.1. `goto`

```
for (const File& text_file : files) {
    for (std::string_view line : text_file.lines()) {
        if (makes_me_angry(line)) {
            goto done_with_file;
        }
        consume(line);
    }
    std::println("Processed {}", text_file.path());
    done_with_file:
}
std::println("Processed all files");
```

`goto` is similar in complexity and even readability here, however there are some issues:

- `goto` cannot cross (non-vacuous) initialization, which would be an issue if some variable was initialized prior to `std::println`. This can be addressed by surrounding the outer loop contents with another set of braces, but this solution isn't obvious and takes away from the elegance of `goto` here.
- `goto` cannot be used in constant expressions. For processing text files like in the example, this doesn't matter, but nested loops are desirable in a `constexpr` context as well.
- Many style guides ban or discourage the use of `goto`. See [MISRA-C++], [CppCoreGuidelinesES76], etc. This discouragement dates all the way back to 1968 (see [GotoConsideredHarmful]), and 66 years of teaching not to use `goto` won't be undone.
- Even in the cases where `goto` isn't discouraged, those cases are always special, like "only `goto` forwards", "only `goto` to break out of loops", etc.. This issue has been debated for decades, and there is still no consensus on when, actually, `goto` is okay to use.
- `goto` is innately more difficult to use because to understand its purpose, the user has to know where the jump target is located. A `goto past_the_loop` behaves radically differently compared to a `goto before_the_loop`. Moving the jump target or the `goto` statement relative to each other can also completely change these semantics. By comparison, `break` and `continue` always jump forwards, past a surrounding loop, or to the end of a surrounding loop respectively. This makes them much easier to reason about, and much less error-prone.
- The "local readability" of `goto` relies heavily on high-quality naming for the label. A `goto end` could mean to the end of a loop, to after the loop, to the end of a function, etc. Since `break` and `continue` are much more limited, they do not require such good label naming. A `break loop` has bad name, but the user *generally* understands its purpose.

> NOTE: Previous discussion on the [isocpp-core] reflector has addressed the idea of just adding `constexpr goto`, but doing so is alleged to be more complicated than more limited `constexpr` control flow structures which can only "jump forwards", such as `break` and `continue`.

In conclusion, there are too many issues with `goto`, some of which may never be resolved. [std-proposals] discussion prior to the publication of this proposal has shown once again that `goto` is a controversial and divisive.

§ 3.1.2. Immediately invoked lambda expression (IILE)

```
for (const File& text_file : files) {
    [&] {
        for (std::string_view line : text_file.lines()) {
```

```
            if (makes_me_angry(line)) {
                return;
            }
            consume(line);
        }
        std::println("Processed {}", text_file.path());
    }();
}
std::println("Processed all files");
```

While this solution works in constant expressions, we may be painting ourselves into a corner with this design. We cannot also `break` the surrounding loop from within the IILE, and we cannot return from the surrounding function. If this is needed at some point, we will have to put substantial effort into refactoring.

Furthermore, this solution isn't exactly elegant:

- The level of indentation has unnecessarily increased through the extra scope.
- The call stack will be one level deeper during debugging. This may be relevant to debug build performance.
- The fact that the lambda is immediately invoked isn't obvious until reading up to `()`.
- The word `return` does not express the overall intent well, which is merely to continue the outer loop. This can be considered a teachability downside.

It is also possible to use an additional function instead of an IILE in this place. However, this is arguably increasing the degree of complexity even more, and it scatters the code across multiple functions without any substantial benefit.

§ 3.1.3. Mutable `bool` state

```
for (const File& text_file : files) {
    bool success = true;
    for (std::string_view line : text_file.lines()) {
        if (makes_me_angry(line)) {
            success = false;
            break;
        }
        consume(line);
    }
```

```
    if (success) {
        std::println("Processed {}", text_file.path());
    }
}
std::println("Processed all files");
```

This solution substantially increases complexity. Instead of introducing extra scope and call stack depth, we add more mutable state to our function. The original intent of "go process the next file" is also lost.

Such a solution also needs additional state for *every* nested loop, i.e. N `bool`s are needed to `continue` from N nested loops.

§ 3.2. Constant expressions

Use of `constexpr` has become tremendously more common, and `goto` may not be used in constant expressions. Where `goto` is used to break out of nested loops, `break label` makes it easy to migrate code:

> EXAMPLE 2 ¶
> Uses of `goto` to break out of nested loops can be replaced with `break label` as follows:
>
> ```
> constexpr void f() {
>     outer: while (/* ... */) {
>         while (/* ... */) {
>             if (/* ... */) {
>                 goto after_loop;
>                 break outer;
>             }
>         }
>     }
>     after_loop:;
> }
> ```

Due to reasons mentioned above, I do not believe that "`constexpr goto`" is a path forward that will find consensus.

§ 3.3. Argumentum ad populum

Another reason to have `break label` and `continue label` is simply that it's a popular construct, available in other languages. When Java, JavaScript, Rust, or Kotlin developers pick up C++, they may expect that C++ can `break` out of nested loops as well, but will find themselves disappointed.

[StackOverflow] "*Can I use break to exit multiple nested `for` loops?*" shows that there is interest in this feature (393K views at the time of writing).

A draft of the proposal was posted on [Reddit] and received overwhelmingly positive feedback (70K views, 143 upvotes with, 94% upvote rate at the time of writing).

§ 3.3.1. Poll

Another way to measure interest is to simply ask C++ users. The following is a committee-style poll (source: [TCCPP]) from the Discord server Together C & C++, which is the largest server in terms of C++-focused message activity:

> Should C++ have "break label" and "continue label" statements to apply break/continue to nested loops or switches?
>
SF	F	N	A	SA
21	21	12	6	4

> NOTE: 64 users in total voted, and the poll was active for one week.

§ 3.3.2. How common is `break`/`continue` with labels?

To further quantify the popularity, we can use GitHub code search for various languages which already support this feature. The following table counts only control statements with a label, *not* plain `break;`, `continue;`, etc. We also count statements like Perl's `last label`; it is de-facto `break label`, just with a different spelling.

Language	Syntax	Labeled `break`s	Labeled `continue`s	Σ `break` `continue`

Java	`label: for (...)`
	`break label;`
	`continue label;`
		424K files	152K files	576K files

JavaScript	`label: for (...)`
	`break label;`
	`continue label;`
		53.8K files	68.7K files	122.5K files

Perl	`label: for (...)`
	`last label;`
	`next label;`
		34.9K files	31.7K files	66.6K files

Rust	`label: for (...)`
	`break 'label;`
	`continue 'label;`
		30.6K files	29.1K files	59.7K files

TypeScript	`label: for (...)`
	`break label;`
	`continue label;`
		11.6K files	9K files	20.6K files

Swift	`label: for ...`
	`break label`
	`continue label`
		12.6K files	5.6K files	18.2K files

Kotlin	`label@ for (...)`
	`break@label`
	`continue@label`
		8.7K files	7.6K files	16.3K files

D	`label: for (...)`
	`break label;`
	`continue label;`
		3.5K files	2.6K files	6.1K files

Go	`label: for ...`
	`break label;`
	`continue label;`
		270 files	252 files	522

Ada	`label: for ...` `exit label;`	N/A	N/A	N/A

Dart	`label: for ...`
	`break label;`
	`continue label;`
		N/A	N/A	N/A

Cpp2 (cppfront)	`label: for ...`
	`break label;`
	`continue label;`
		N/A	N/A	N/A

C	`label: for (...)`
	`break label;`
	`continue label;`
		N/A	N/A	N/A

Fortran	`label: do ...` `exit label`	N/A	N/A	N/A

Groovy	`label: for ...`
	`break label;`
	`continue label;`
		N/A	N/A	N/A

Odin	`label: for ...` `break label;`	N/A	N/A	N/A
PL/I	`label: do ...` `exit label;`	N/A	N/A	N/A
PostgreSQL	`<<label>> for ...` `exit label;`	N/A	N/A	N/A
PowerShell	`:label for ...` `break outer`	N/A	N/A	N/A

Based on this, we can reasonably estimate that there are at least one million files in the world which use labeled `break`/`continue` (or an equivalent construct).

> NOTE: This language list is not exhaustive and the search only includes open-source code bases on GitHub. Some of the cells are N/A because the number isn't meaningful, or simply because I haven't gotten around to doing the code search yet.

§ 3.4. Expansion statements

The proposed `break label` feature also has interesting interactions with C++26 expansion statements.

EXAMPLE 3 ¶

`break label` makes it possible to `break` out of a loop that the expansion statement is nested within.

```
outer: for (const auto& tuple : list_of_tuples) {
    template for (const auto& e : tuple) {
        if (/* ... */) {
            break outer;
        }
    }
}
```

A regular `break;` statement would instead apply to the expansion statement itself, meaning that neither `break` nor `continue` of loops is possible from expansion statements.

EXAMPLE 4 ¶

In the future, it may be possible to programmatically generate `case`s for a `switch` using expansion statements. If so, `break label` is needed to prevent fallthrough between cases:

```
outer: switch (/* ... */) {
    template for (constexpr int N : cases) {
        case N: f<N>();
        break outer;
    }
}
```

Without the `break outer`, the generated code would be:

```
case 0: f<0>(); // fallthrough!
case 1: f<1>();
case 2: f<2>();
```

§ 3.5. C2y compatibility

Last but not least, C++ should have `break label` and `continue label` to increase the amount of code that has a direct equivalent in C. Such compatibility is desirable for two reasons:

- `inline` functions or macros used in C/C++ interoperable headers could use the same syntax.
- C2y code is much easier to port to C++ (and vice-versa) if both languages support the same control flow constructs.

Furthermore, the adoption of [N3355] saves EWG a substantial amount of time when it comes to debating the syntax; the C++ syntax should certainly be C-compatible.

4. Design Considerations

4.1. Alternative `break` and `continue` forms

While the idea of applying `break` and `continue` to some surrounding construct of choice is simple, there are infinite ways to express this. Various ideas have been proposed over the last months and years:

- `break 1`, `break 2`, ... (specify the amount of loops, not the targeted loop)
- `break while`, `break for while`, ... (target by keyword, not by label)
- `break break` (execute statement in the jumped-to scope)
- `for name (...)` (competing syntax in [N3377])

All of these have been discussed in great detail in the first revision of this paper, [P3568R0]. At this point, it would be a waste of time to discuss these in detail.

WG21 *overwhelmingly* agrees (based on polls, reflector discussions, and personal conversations) that the design should be compatible with C. This is also reflected by a poll at Hagenberg 2025:

> **P3568R0**: If C has it, we are interested in this feature too.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 16 | 21 | 5 | 2 | 1 |

Furthermore, WG14 has already accepted the `label: for` syntax of [N3355] into C2y, and WG14 is unwilling to revisit this syntax, as voted at Graz 2025:

> **N3377**: Would WG14 like to see a paper changing loop name syntax at a future meeting?
>
> | F | N | A |
> |---|---|---|
> | 6 | 11 | 9 |

There is *only* one way forward that has a chance of finding consensus: **do what C does.**

§ 4.2. Changes to labels

While the proposed `for name (...)` syntax of [N3377] was de-facto rejected at Graz, the paper brings up legitimate issues with C2y `break label` after [N3355].

Notably, the restriction that a label can be used only once per function is not usually present in other languages that support `break label`. This restriction is especially bad for C and C++ because if `label:` was used in a macro, that macro could only be expanded once per function:

```
#define MACRO() outer: for (/* ... */) for (/* ... */) break outer;

void f() {
    MACRO() // OK so far
    MACRO() // error: duplicate label 'outer'
}
```

The author of [N3355] has expressed to me that he intends to address these label issues for C2y. In parallel, this proposal addresses such issues by relaxing label restrictions. Presumably, C and C++ will converge on identical restrictions.

§ 4.2.1. Design philosophy

The proposed design is extremely simple:

> 1. Drop *all* restrictions on labels.
> 2. Make `break label` and `continue label` "just work" anyway.
> 3. Disallow `goto label` for duplicate `label`.

Any existing `goto` code remains unaffected by this change. These rules are simple and easy to remember.

While it may seem too lax to put no restrictions on labels at all, there's no obvious problem with this. Labels don't declare anything, and unless referenced by `break` and `goto`, they are de-facto comments with zero influence on the labeled code. If labels are quasi-comments, why should there be any restrictions on the labels themselves?

The consequences and details of these changes are described below.

§ 4.2.2. Allowing duplicate labels

I propose to permit duplicate labels, which makes the following code valid:

```
outer: while (true) {
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

outer: while (true) { // OK, reusing label is permitted
    inner: while (true) {
        break outer; // breaks enclosing outer while loop
    }
}

goto outer; // error: ambiguous jump target
```

Such use of duplicate labels (possibly with different syntax) is permitted in numerous other languages, such as Rust, Kotlin, Java, JavaScript, TypeScript, Dart, and more. To be fair, languages that also support `goto` require unique labels per function, but there's no technical reason why the uniqueness restriction couldn't be placed on `goto` rather than the labels themselves.

As mentioned before, permitting such code is especially useful when these loops are not hand-written, but expanded from a macro. Even disregarding macros, there's nothing innately wrong about this code, and it is convenient to reuse common names like `outer:` for controlling nested loops.

> NOTE: Existing code using `goto` is unaffected because existing code cannot have duplicate labels in the first place.

§ 4.2.3. Reusing labels in nested loops

A more controversial case is the following:

```
l: while (true) {
    l: while (true) {
        break l; // equivalent to break;
    }
}
```

`break l` generally applies to the innermost loop labeled `l:`, so the inner loop is targeted here. I believe that this code should be valid because it keeps the label restrictions stupidly simple (there are none), and because this feature may be useful to developers.

One may run into this case when nesting pairs of `outer:`/`inner:` loops in each other "manually", or when an `l:`-labeled loop in a macro is expanded into a surrounding loop that also uses `l:`.

> NOTE: This code is not valid Java or JavaScript, but is valid Rust when using the label `'l`.

§ 4.2.4. Duplicate labels on the *same* statement

A more extreme form of the scenario above is:

```
l: l: l: l: f();
```

I also believe that this code should be valid because it's not harmful, and may be useful in certain, rare situations (see below). Once again, allowing it keeps the label restrictions stupidly simple.

```
// common idiom on C: expand loops from macros
#define MY_LOOP_MACRO(...) outer: for (/* ... */)

outer: MY_LOOP_MACRO(/* ... */) {
    break outer;
}
```

If `MY_LOOP_MACRO` already uses an `outer:` label internally, perhaps because it expands to two nested loops and uses `continue outer;` itself, then the macro effectively expands to `outer: outer:`. This forces the user to come up with a new label now, for no apparent reason.

4.2.5. `break label` for loops with more than one label

Another case to consider is this:

```
x: y: while (true) {
    break x; // OK in C2y
}
```

[N3355] makes wording changes to C so that the code above is valid. For C2y compatibility and convenience, we also make this valid. We don't change the C++ grammar to accomplish this, but define the term *(to) label (a statement)*, where `x` labels `while`.

5. Impact on existing code

No existing code becomes ill-formed or has its meaning altered. This proposal merely permits code which was previously ill-formed, and relaxes restrictions on the placement of labels.

6. Implementation experience

An LLVM implementation is W.I.P.

A GCC implementation of [N3355] has also been committed at [GCC].

7. Proposed wording

The wording is relative to [N5032] with changes from the 2026-03 Croydon meeting applied.

Update [stmt.label] paragraph 1 as follows:

A label can be added to a statement or used anywhere in a *compound-statement*.

label:

attribute-specifier-seq$_{\text{opt}}$ *identifier* `:`

attribute-specifier-seq$_{\text{opt}}$ `case` *constant-expression* `:`

attribute-specifier-seq$_{\text{opt}}$ `default :`

labeled-statement:

label statement

The optional *attribute-specifier-seq* appertains to the label. ~~The only use of a label with an *identifier* is as the target of a `goto`. No two labels in a function shall have the same identifier.~~ A label can be used in a `goto` statement ([stmt.goto]) before its introduction.

[*Note*: Multiple identical labels within the same function are permitted, but such duplicate labels cannot be used in a `goto` statement. — *end note*]

In [stmt.label] insert a new paragraph after paragraph 1:

A label *L* of the form *attribute-specifier-seq*$_{\text{opt}}$ *identifier* `:` *labels* the *statement S* of a *labeled-statement X* if

- *L* is the *label* of *X*, or
- *L* labels *X* (recursively).

[*Example*:

```
a: b: while (0) { }            // both a: and b: label the loop
c: { d: switch (0) {           // unlike c:, d: labels the switch stateme
    default: while (0) { }     // default: labels nothing
} }
```

— *end example*]

NOTE: This defines the term *(to) label*, which is used extensively below. We also don't want `case` or `default` labels to label statements, since this would inadvertently permit `break i` given `case i:`, considering how we word [stmt.break].

Update [stmt.label] paragraph 3 as follows:

> A *control-flow-limited statement* is a statement *S* for which:
>
> - a `case` or `default` label appearing within *S* shall be associated with a `switch` statement ([stmt.switch]) within *S* , and
> - a label declared in *S* shall only be referred to by a statement ~~([stmt.goto])~~ in *S* .

NOTE: While the restriction still primarily applies to `goto` (preventing the user from e.g. jumping into an `if constexpr` statement), if other statements can also refer to labels, it is misleading to say "statement ([stmt.goto])" as if `goto` was the only relevant statement.

Update [stmt.jump.general] paragraph 1 as follows:

> Jump statements unconditionally transfer control.
>
> *jump-statement*:
>
> `goto` *identifier* `;`
>
> `break` *identifier*$_{\text{opt}}$ `;`
>
> `continue` *identifier*$_{\text{opt}}$ `;`
>
> `return` *expr-or-braced-init-list*$_{\text{opt}}$ `;`
>
> *coroutine-return-statement* ~~`goto` *identifier* `;`~~

NOTE: `goto` is being relocated to the top so that all the jump statements with an *identifier* are grouped together. Of these three, `goto` is being listed first because it models the concept of "jumping somewhere" most literally; every following statement is more sophisticated or even defined as equivalent to `goto` (in the case of `continue`).

Update [stmt.break] paragraph 1 as follows:

A *breakable statement* is an *iteration-statement* ([stmt.iter]), an *expansion-statement*, or a `switch` statement ([stmt.switch]). A `break` statement shall be enclosed by ([stmt.pre]) ~~an *iteration-statement* ([stmt.iter]), an *expansion-statement*, or a `switch` statement ([stmt.switch])~~ a breakable statement . If present, the *identifier* shall be part of a label *L* which labels ([stmt.label]) an enclosing breakable statement. The `break` statement causes termination of : ~~the smallest such enclosing statement;~~

- if an *identifier* is present, the smallest enclosing breakable statement labeled by *L* ,
- otherwise, the smallest enclosing breakable statement.

~~control~~ Control passes to the statement following the terminated statement, if any.

[*Example*:

```
a: b: while (/* ... */) {
    a: a: c: for (/* ... */) {
        break;              // OK, terminates enclosing for loop
        break a;            // OK, same
        break b;            // OK, terminates enclosing while loop
        y: { break y; }     // error: break does not refer to a breakable
    }
    break c;                // error: break does not refer to an enclosi
}
break;                      // error: break is not enclosed by a breakab
```

— end example]

✧ ✧ ✧

Update [stmt.cont] paragraph 1 as follows:

> A *continuable statement* is an *iteration-statement* ([stmt.iter]) or an *expansion-statement*. A `continue` statement shall be enclosed by ([stmt.pre]) ~~an *iteration-statement* or *expansion-statement*~~ a continuable statement . If present, the *identifier* shall be part of a label L which labels ([stmt.label]) an enclosing continuable statement. The `continue` statement applies to a statement X, which is:
>
> - if an *identifier* is present, the smallest enclosing continuable statement labeled by L,
> - otherwise, the smallest enclosing continuable statement.
>
> If the innermost enclosing such statement X is an *iteration-statement* ([stmt.iter]), the `continue` statement causes control to pass to the end of the *statement* of X . Otherwise, control passes to the end of the *compound-statement* of the current X_i .

✧ ✧ ✧

Update [stmt.goto] paragraph 1 as follows:

> The `goto` statement unconditionally transfers control to ~~the~~ a statement labeled ([stmt.label]) by ~~the identifier~~ a *label* in the current function containing *identifier* . ~~The identifier shall be a label ([stmt.label]) located in the current function.~~ There shall be exactly one such label.

NOTE: This wording has always been defective and our proposal fixes this. The term "to label" was never defined, and the requirement that an identifier shall be a label is impossible to satisfy because a label ends with a `:`, and an *identifier* in itself would never match the *label* rule.

✧ ✧ ✧

Add a feature-test macro to [tab:cpp.predefined.ft] as follows:

> | Macro name | Value |
> |---|---|
> | `__cpp_break_label` | `20????L` |

§ 8. Acknowledgements

I thank Sebastian Wittmeier for providing a list of languages that support both `goto` and `break`/`last` with the same label syntax.

I think Arthur O'Dwyer and Jens Maurer for providing wording feedback and improvement suggestions.

I especially thank Arthur O'Dwyer for helping me expand the list in § 3.3.2 How common is break/continue with labels?. An even more complete list may be available at [ArthurBlog].

I thank the Together C & C++ community for responding to my poll; see [TCCPP].

I thank Matthias Wippich for explaining the interactions of `break label` with expansion statements; see § 3.4 Expansion statements.

§ References

§ Normative References

[N5032]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025. URL: https://wg21.link/n5032

§ Informative References

[ArthurBlog]
Arthur O' Dwyer. *Arthur O' Dwyer*. URL: https://quuxplusone.github.io/blog/2024/12/20/labeled-loops/

[CppCoreGuidelinesES76]
CppCoreGuidelines contributors. *CppCoreGuidelines/ES.76: Avoid goto*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-goto

[GCC]
Jakub Jelinek. *c: Implement C2Y N3355 - Named Loops [PR117022]*. URL: https://gcc.gnu.org/git/gitweb.cgi?p=gcc.git;h=50f27896adb272b40ab03a56fd192e74789bef97

[GotoConsideredHarmful]
Edgar Dijkstra. *Go To Statement Considered Harmful*. 1968. URL: https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf

[ISOCPP-CORE]

CWG. *Discussion regarding continue vs. goto in constant expressions*. URL: https://lists.isocpp.org/core/2023/05/14228.php

[MISRA-C++]

MISRA Consortium Limited. *MISRA C++:2023*. URL: https://misra.org.uk/product/misra-cpp2023/

[N3355]

Alex Celeste. *N3355: Named loops, v3*. 2024-09-18. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3355.htm

[N3377]

Erich Keane. *N3377: Named Loops Should Name Their Loops: An Improved Syntax For N3355*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3377.pdf

[N3435]

JeanHeyd Meneide; Freek Wiedijk. *ISO/IEC 9899:202y (en) — n3435 working draft*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3435.pdf

[N3879]

Andrew Tomazos. *Explicit Flow Control: break label, goto case and explicit switch*. 16 January 2014. URL: https://wg21.link/n3879

[N4327]

Ville Voutilanen. *C++ Standard Evolution Closed Issues List (Revision R10)*. 21 November 2014. URL: https://wg21.link/n4327

[P3568R0]

Jan Schultke, Sarah Quiñones. *break label; and continue label;*. 12 January 2025. URL: https://wg21.link/p3568r0

[Reddit]

Jan Schultke. *"break label;" and "continue label;" in C++*. URL: https://www.reddit.com/r/cpp/comments/1hwdskt/break_label_and_continue_label_in_c/

[StackOverflow]

Faken. *Can I use break to exit multiple nested 'for' loops?*. 10 Aug 2009. URL: https://stackoverflow.com/q/1257744/5740428

[STD-PROPOSALS]

Jan Schultke. *Bringing break/continue with label to C++*. URL: https://lists.isocpp.org/std-proposals/2024/12/11838.php

[TCCPP]

Poll at Together C & C++ (discord.gg/tccpp). URL: https://discord.com/channels/331718482485837825/851121440425639956/1318965556128383029